SOFTWARE ENGINEERING — UNIT 3

Complete Semester Exam Notes • 6-Mark Answer Format

SOFTWARE ENGINEERING — UNIT 3

Complete Semester Exam Notes (6-Mark Answers)

Your Unit 3 syllabus contains three major sections:

- 1. **Software Architecture and Object-Oriented Design**
- 2. **UML (Unified Modeling Language)**
- 3. **Interaction Diagrams**

Each topic is explained in simple language, with definitions, important points, examples, and diagrams you can reproduce in your semester examination.

PART 1: SOFTWARE ARCHITECTURE

1. Software Architecture and Architecture Views

Definition

Software architecture is the **high-level structure of a software system**. It describes the major components of a system, their responsibilities, and how they communicate with one another.

Think of constructing a house. Before building it, an architect prepares a blueprint. Similarly, software architecture provides a blueprint before developers implement the complete software.

Importance of Software Architecture

- 1. **High-level design:** Describes the overall structure of the system.
- 2. **Communication:** Helps developers understand how different components interact.
- 3. **Maintainability:** Makes it easier to modify individual components.
- 4. **Performance:** Helps identify performance requirements and bottlenecks.
- 5. **Reusability:** Allows components to be reused in different applications.
- 6. **Scalability:** Helps the system accommodate increasing workloads.

Architecture Views

An architecture view represents a particular aspect of the software system.

A commonly taught architectural model is the **4+1 View Model**.

It contains four main views and one additional view based on scenarios.

1. Logical View

• Describes the important classes, objects, and their relationships.

• Focuses on the functionality provided by the system.

• Useful for developers and designers.

Example: Classes such as Student, Course, and Faculty in a college management system.

2. Process View

• Describes processes and activities occurring during program execution.

• Focuses on concurrency, communication, and performance.

• Helps understand how different processes work together.

Example: Multiple users accessing a college portal simultaneously.

3. Development View

• Describes how software is organized into modules, packages, and components.

• Helps developers understand the code structure.

• Supports software development and maintenance.

Example: Separating a project into frontend, backend, and database modules.

4. Physical View

• Describes how software components are deployed on hardware.

• Shows servers, computers, and network connections.

• Helps understand the deployment structure.

Example: A React frontend hosted on Vercel communicating with a backend hosted on Render.

5. Scenario View (+1)

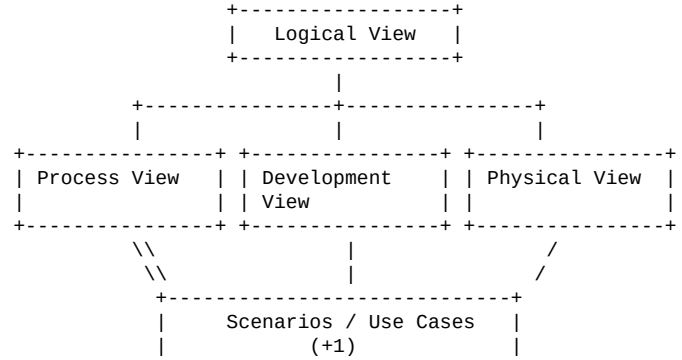
• Represents important use cases or scenarios.

• Demonstrates how the other four views work together.

• Helps validate the architecture against user requirements.

Example: A student logs in, checks attendance, and downloads a report.

Diagram: 4+1 View Model



```
+----------------------------+
```

Conclusion: Architecture views describe a software system from different perspectives, making it easier to design, develop, deploy, and maintain.

Exam question: Explain software architecture and its different architectural views.

2. Architectural Styles for Component-and-Connector (C&C) Views

Definition

A **Component-and-Connector (C&C) view** describes the runtime components of a software system and the connections through which they communicate.

It consists of two main elements.

Components: Independent processing units or runtime entities.

Examples: Clients, servers, filters, and databases.

Connectors: Mechanisms that allow components to communicate.

Examples: Network requests, pipes, messages, and shared-data access.

Architectural Styles

An architectural style defines a standard way of organizing components and their interactions.

Your syllabus includes three important styles:

• 1. Pipe-and-Filter

• 2. Client-Server

• 3. Shared Data

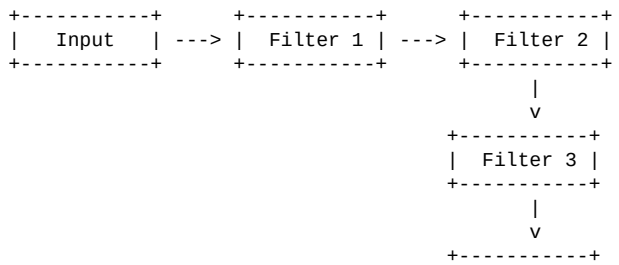
3. Pipe-and-Filter Architectural Style

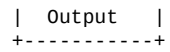
Definition

The Pipe-and-Filter architectural style organizes a system as a sequence of processing components called **filters**. Each filter receives input data, processes it, and produces output data.

The output of one filter becomes the input of another through a **pipe**.

Diagram





Components

1. Filters

• Perform specific data-processing tasks.

• Receive input and produce output.

• Can often operate independently of other filters.

2. Pipes

• Transfer data between filters.

• Connect the output of one filter to the input of another.

Working

Consider a system that processes student examination results.

• 1. The input filter reads students' marks.

• 2. The second filter calculates total marks.

• 3. The third filter calculates percentages.

• 4. The final filter determines grades and produces the results.

Each filter performs a separate operation.

Advantages

• 1. Filters can be reused in different processing sequences.

• 2. Individual filters can be modified independently.

• 3. The system is easier to test and maintain.

• 4. Some filters can execute in parallel.

• 5. New processing stages can be added when needed.

Disadvantages

• 1. Data conversion between filters may introduce overhead.

• 2. The system may be inefficient for interactive applications.

• 3. A slow filter can delay the entire pipeline.

• 4. Filters must agree on the format of the data they exchange.

Applications

• Compiler processing stages.

• Image processing.

• Data transformation systems.

• Audio and video processing.

• Unix command pipelines.

Example: A compiler can perform lexical analysis, syntax analysis, semantic analysis, and intermediate-code generation in successive stages.

Exam question: Explain the Pipe-and-Filter architectural style with a neat diagram.

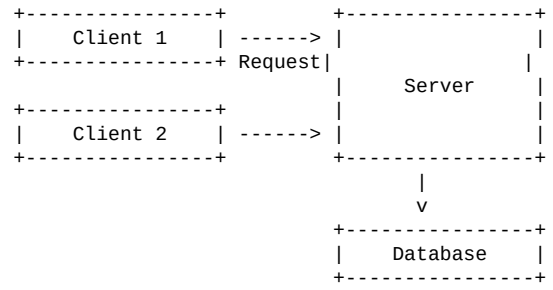
4. Client-Server Architectural Style

Definition

The Client-Server architectural style divides a system into clients that request services and servers that provide those services.

A client sends a request to a server, the server processes the request, and the server returns a response.

Diagram



Components

1. Client

• Requests services or resources.

• Presents information to users.

• Sends requests to the server.

2. Server

• Receives and processes client requests.

• Performs business logic.

• Returns the required response.

3. Network

• Provides communication between clients and servers.

4. Database

• Stores and retrieves application data when required.

Working

Consider an online shopping application.

• 1. A customer opens the application.

• 2. The client sends a request for product information.

• 3. The server receives the request.

• 4. The server retrieves product details from the database.

• 5. The server sends the product information back to the client.

• 6. The client displays the products.

Advantages

• 1. Centralized management of data.

• 2. Easier maintenance and administration.

• 3. Multiple clients can access the same server.

• 4. Security policies can be managed centrally.

• 5. Server resources can be upgraded independently.

Disadvantages

• 1. Server failure may interrupt services.

• 2. Network problems can prevent communication.

• 3. Heavy traffic may overload the server.

• 4. Security vulnerabilities in the server can affect many clients.

Applications

• Online banking.

• Web applications.

• Email services.

• Online shopping.

• College management systems.

Exam question: Explain the Client-Server architectural style with a suitable diagram.

5. Shared-Data Architectural Style

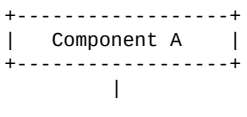
Definition

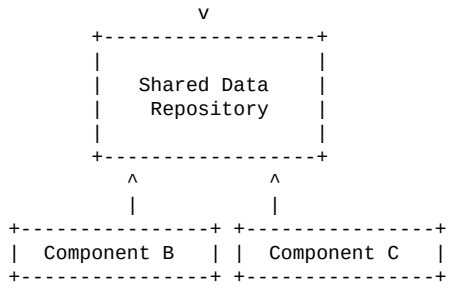
The Shared-Data architectural style organizes a software system around a **common data store** that multiple independent components can access.

The components communicate indirectly by reading from or writing to the shared data store.

It is also commonly called a **repository style** in many software architecture contexts.

Diagram





Working

Consider a college management system.

• 1. The student module stores student details in the repository.

• 2. The examination module retrieves student details from the same repository.

• 3. The attendance module updates attendance records.

• 4. The report-generation module reads the stored information to generate reports.

All components use the common data store.

Advantages

• 1. Provides a centralized source of data.

• 2. Reduces duplication of information.

• 3. Allows multiple components to share information.

• 4. Makes data management easier.

• 5. Components can be developed independently if they follow the repository's interface and data rules.

Disadvantages

• 1. Failure of the central repository can affect the entire system.

• 2. Multiple simultaneous updates may create concurrency problems.

• 3. Changes to the repository schema may affect several components.

• 4. A heavily used repository can become a performance bottleneck.

• 5. Access to shared information must be controlled carefully.

Applications

• Database management systems.

• Library management systems.

• College management systems.

• Data warehouses.

• Systems in which multiple tools share a common data model.

Difference Between the Three Architectural Styles

Feature	Pipe-and-Filter	Client-Server	Shared Data
Main idea	Data passes through processing stages	Clients request services from servers	Components access a common data store
Communication	Through pipes	Through requests and responses	Through shared storage
Main components	Filters and pipes	Clients and servers	Components and repository
Main advantage	Reusability of processing stages	Centralized services	Consistent, shared data
Example	Compiler pipeline	Web application	College database system

Important exam tip: Prepare all three styles with a definition, diagram, working, advantages, disadvantages, and applications.

PART 2: OBJECT-ORIENTED DESIGN

6. Objects and Object Classes

Definition of Object

An **object** is an entity that has state, behavior, and identity.

• **State:** The data or properties of the object.

• **Behavior:** The operations the object performs.

• **Identity:** The feature that distinguishes one object from another.

Example

Consider a student object.

```
Student
-------------------------
Roll Number: 101
Name: Ravi
Marks: 85
-------------------------
calculateGrade()
displayDetails()
```

Here:

• Roll number, name, and marks represent the state.

• `calculateGrade()` and `displayDetails()` represent behavior.

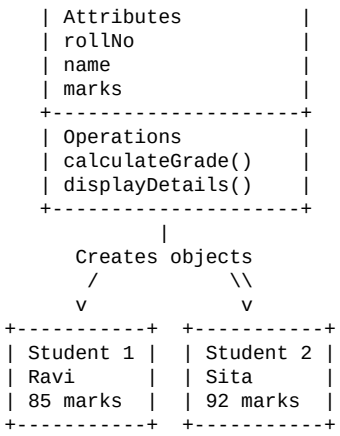
• The student object has its own identity.

Definition of Object Class

An **object class** is a blueprint that defines the common attributes and operations of a group of objects.

For example, a Student class defines the properties and operations shared by student objects.

```
            Student Class
          +--------------------+
```



Difference Between Object and Class

Object	Class
An instance of a class	A blueprint for objects
Has its own state	Defines the attributes
Can perform operations	Defines the available operations
Occupies memory when instantiated	A class definition describes the structure
Example: Student Ravi	Example: Student class

Main Characteristics of Object-Oriented Design

• 1. **Encapsulation:** Combines data and operations in one unit.

• 2. **Abstraction:** Hides unnecessary implementation details.

• 3. **Inheritance:** Allows a class to reuse or extend another class's features.

• 4. **Polymorphism:** Allows the same operation or interface to behave differently in different contexts.

Conclusion: Objects represent individual entities, while classes define the common structure and behavior of those entities.

7. Object-Oriented Design Process

Definition

Object-Oriented Design (OOD) is the process of designing software by identifying objects, classes, their responsibilities, and their relationships.

It transforms system requirements into a design that developers can implement using an object-oriented programming language.

Steps in the Object-Oriented Design Process

Step 1: Understand the Requirements

Study the Software Requirements Specification (SRS).

Identify the required functionality and constraints.

Example: For a library system, users should be able to search, borrow, and return books.

Step 2: Identify Objects and Classes

Identify the important entities in the system.

Example:

• Book

• Member

• Librarian

• Loan

Step 3: Identify Attributes and Operations

Determine the data and functions associated with each class.

For example:

Book attributes: Book ID, title, author.

Book operations: Search book, check availability.

Step 4: Establish Relationships

Identify how the classes are connected.

Examples:

• A Member borrows a Book.

• A Librarian manages Books.

• A Loan records a borrowing transaction.

Step 5: Define Interfaces and Responsibilities

Specify what each class does and how other classes interact with it.

For example, a Loan class manages borrowing dates and return dates.

Step 6: Create UML Diagrams

Prepare suitable UML diagrams, such as:

• Class diagrams.

• Use case diagrams.

• Sequence diagrams.

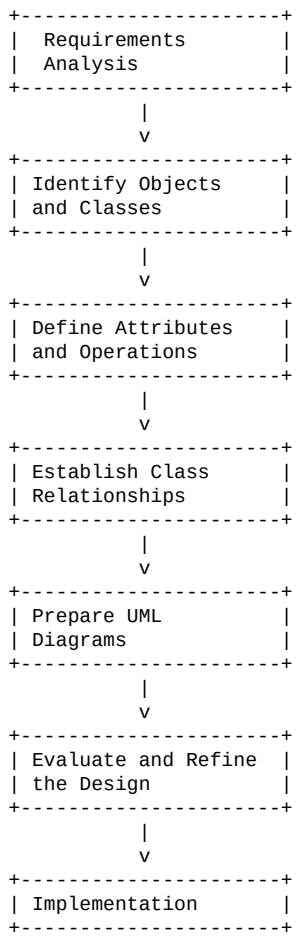
These diagrams help communicate and validate the design.

Step 7: Evaluate and Refine the Design

Check whether the design satisfies the requirements.

Improve class responsibilities, remove unnecessary dependencies, and address design problems.

Diagram: Object-Oriented Design Process



Advantages

- 1. Promotes modularity.
- 2. Encourages code reuse.
- 3. Improves maintainability.
- 4. Makes complex systems easier to understand.
- 5. Supports changes in requirements.

Exam question: Explain the Object-Oriented Design process with a diagram.

8. Design Evolution

Definition

Design evolution is the process of modifying and improving a software design as requirements, technologies, or business needs change.

Software design is not always fixed permanently after the first version. It evolves throughout the software life cycle.

Reasons for Design Evolution

• 1. **Changing requirements:** Users may request new features.

• 2. **Bug correction:** Design changes may be required to correct defects.

• 3. **Performance improvement:** Components may need optimization.

• 4. **Technology changes:** New frameworks or platforms may require modifications.

• 5. **Maintainability:** Complex components may need restructuring.

• 6. **Scalability:** The system may need to support more users or data.

Example

Consider an online shopping system.

Initial design:

The system supports product browsing and ordering.

First evolution:

A payment module is introduced.

Second evolution:

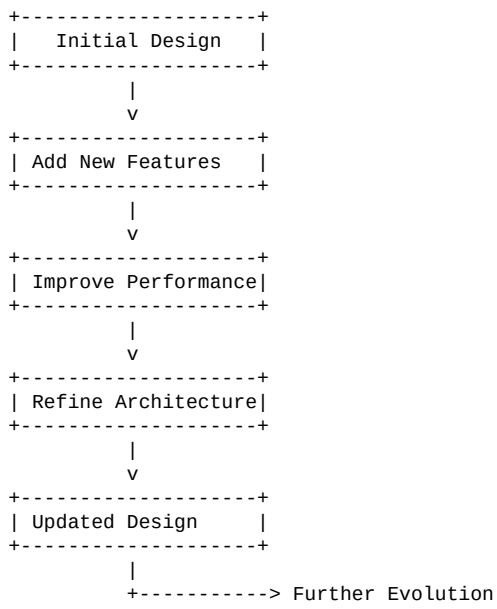
A delivery-tracking module is added.

Third evolution:

The system is redesigned to handle more users and larger product inventories.

The design changes as new requirements arise.

Diagram



Important Techniques

• **Refactoring:** Improving internal code structure without changing its external behavior.

• **Redesign:** Modifying the design to satisfy new or changed requirements.

• **Incremental development:** Adding functionality in small stages.

• **Design evaluation:** Checking whether the updated design still satisfies requirements.

Advantages

• 1. Keeps software aligned with changing needs.

• 2. Improves quality and maintainability.

• 3. Allows new functionality to be added.

• 4. Helps address performance problems.

• 5. Extends the useful life of software.

Conclusion: Design evolution helps software remain useful, maintainable, and adaptable throughout its lifetime.

PART 3: OVERVIEW OF UML

9. What Is UML?

Definition

UML (Unified Modeling Language) is a standardized visual modeling language used to specify, visualize, construct, and document the structure and behavior of software systems.

UML helps developers represent software designs using diagrams instead of relying entirely on written descriptions.

Objectives of UML

• 1. Visualize software structure and behavior.

• 2. Communicate design decisions clearly.

• 3. Document software systems.

• 4. Represent requirements and interactions.

• 5. Support object-oriented analysis and design.

Types of UML Diagrams

UML diagrams are broadly divided into two categories.

1. Structural diagrams

These describe the static structure of a system.

Examples:

• Class diagram.

• Object diagram.

• Component diagram.

• Deployment diagram.

• Package diagram.

2. Behavioral diagrams

These describe system behavior and interactions.

Examples:

• Use case diagram.

• Activity diagram.

• State machine diagram.

• Sequence diagram.

• Communication diagram.

Your syllabus specifically emphasizes class, use case, sequence, and collaboration diagrams.

10. Building Blocks of UML

The three main building blocks of UML are:

• 1. Things.

• 2. Relationships.

• 3. Diagrams.

A. Things

Things are the basic modeling elements used in UML.

They can be classified into four groups in the traditional UML building-block explanation.

1. Structural Things

Represent the static elements of a system.

Examples:

• Class.

• Interface.

• Component.

• Node.

• Use case.

2. Behavioral Things

Represent behavior over time.

Examples:

• Interactions.

• State machines.

3. Grouping Things

Organize related modeling elements.

Example: Packages group classes and other model elements.

4. Annotational Things

Provide explanatory information about a model.

Example: Notes or comments attached to a UML diagram.

B. Relationships

Relationships connect UML elements.

The major relationships are:

• Dependency.

• Association.

• Generalization.

• Realization.

These are explained in detail in the next section.

C. Diagrams

Diagrams provide a graphical representation of the system.

Examples:

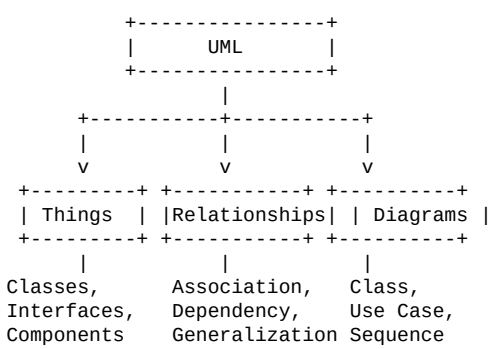
• Class diagram.

• Use case diagram.

• Sequence diagram.

• Collaboration diagram.

Summary Diagram



Exam question: Explain the building blocks of UML.

11. UML Class Diagram

Definition

A class diagram is a structural UML diagram that represents classes, their attributes, operations, and relationships.

It is used to describe the static structure of an object-oriented system.

Structure of a Class

A UML class is usually represented by a rectangle divided into three sections:

• 1. Class name.

• 2. Attributes.

• 3. Operations or methods.

Diagram

```
+-----------------------------+
|           Student           |
+-----------------------------+
| - rollNo : int              |
| - name : String             |
| - marks : double            |
+-----------------------------+
| + calculateGrade() : char   |
| + displayDetails() : void   |
+-----------------------------+
```

Visibility Symbols

Symbol	Meaning
`+`	Public
`-`	Private
`#`	Protected
`~`	Package visibility

For example, `- name : String` represents a private attribute called `name` of type String.

Example: Library Management System

```
+------------------+            +------------------+
|      Member      |            |       Book       |
+------------------+            +------------------+
| - memberId : int |            | - bookId : int    |
| - name : String  |            | - title : String  |
+------------------+            +------------------+
| + borrowBook()   |            | + checkStatus()   |
| + returnBook()   |            | + updateStatus()  |
+------------------+            +------------------+
          |                               |
          +------------- borrows ----------+
```

This shows two classes and their association.

Advantages of Class Diagrams

• 1. Represent the static structure of a system.

• 2. Show relationships between classes.

• 3. Help developers plan class implementations.

• 4. Improve communication among team members.

• 5. Support documentation and maintenance.

Exam question: Draw and explain a UML class diagram with a suitable example.

12. Class Relationships in UML

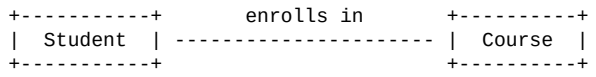
Class relationships explain how classes are connected.

The four fundamental UML relationships are dependency, association, generalization, and realization. Aggregation and composition are important specialized forms of association.

1. Association

Association represents a structural relationship between classes.

Example: A student enrolls in a course.

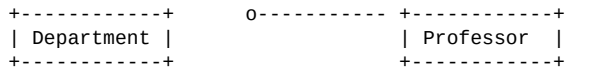


The line represents an association.

2. Aggregation

Aggregation represents a whole-part relationship in which the part can exist independently of the whole.

Example: A department has professors, but professors can exist independently of a particular department.

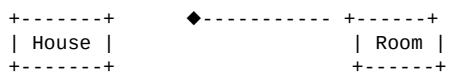


The **hollow diamond (o)** is placed at the whole side.

3. Composition

Composition is a strong whole-part relationship in which the part's lifecycle is tied to the whole in the modeled relationship.

Example: A house contains rooms, where the rooms are treated as integral parts of that particular house model.

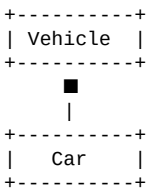


The **filled diamond (◆)** is placed at the whole side.

4. Generalization

Generalization represents inheritance. A specialized class inherits features from a more general class.

Example: A Car is a type of Vehicle.

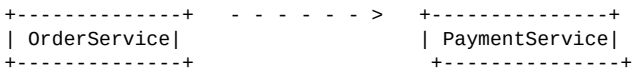


The hollow triangle points toward the general class, `Vehicle`.

5. Dependency

Dependency means one class temporarily uses or depends on another class.

Example: An OrderService uses a PaymentService to process payments.

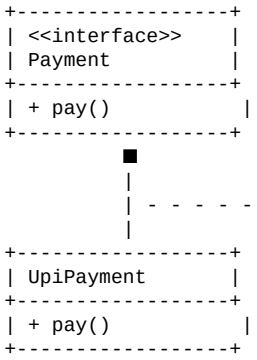


The dashed arrow points toward the class being depended upon.

6. Realization

Realization indicates that a class implements an interface.

Example: A class implements a payment interface.



The dashed line with a hollow triangle points toward the interface.

Quick Revision Table

Relationship	Meaning	UML notation
Association	General structural connection	Solid line
Aggregation	Weak whole-part relationship	Hollow diamond
Composition	Strong whole-part relationship	Filled diamond
Generalization	Inheritance	Solid line with hollow triangle

Relationship	Meaning	UML notation
Dependency	One element uses another	Dashed arrow
Realization	Implementation of an interface	Dashed line with hollow triangle

Exam tip: If asked to explain class relationships, draw at least four notations and give one example for each.

PART 4: USE CASE MODELING

13. Use Case Scenario

Definition

A use case scenario is a description of a particular sequence of interactions between an actor and a system to achieve a specific goal.

A **use case** describes the overall goal or functionality, while a **scenario** describes one particular path through that use case.

Example: ATM Withdrawal

Use case: Withdraw cash.

Actor: Customer.

Precondition: The customer has inserted a valid ATM card.

Main Success Scenario

• 1. The customer inserts the ATM card.

• 2. The ATM requests the PIN.

• 3. The customer enters the PIN.

• 4. The system validates the PIN.

• 5. The customer selects cash withdrawal.

• 6. The customer enters the amount.

• 7. The system checks the account balance.

• 8. The ATM dispenses cash.

• 9. The system updates the account balance.

Alternative Scenario

If the PIN is incorrect, the system displays an error message and asks the customer to try again, subject to its security rules.

Main Elements of a Use Case Scenario

• 1. **Actor:** The person or external system interacting with the application.

• 2. **Goal:** What the actor wants to achieve.

• 3. **Precondition:** What must be true before the use case starts.

• 4. **Main flow:** The normal sequence of steps.

• 5. **Alternative flow:** Another possible path.

• 6. **Postcondition:** The state after the use case finishes.

Applications

• ATM systems.

• Online shopping.

• Library systems.

• Banking applications.

• College portals.

Exam question: Explain a use case scenario with an example.

14. Use Case Diagram

Definition

A use case diagram is a UML behavioral diagram that represents the functionality provided by a system and the external actors who interact with it.

It focuses on **what the system does**, rather than how the internal code implements it.

Main Elements

1. Actor

Represents an external role that interacts with the system.

Notation: A stick figure.

Examples: Student, Admin, Customer.

2. Use Case

Represents a function provided by the system.

Notation: An oval.

Examples: Login, Search Book, Borrow Book.

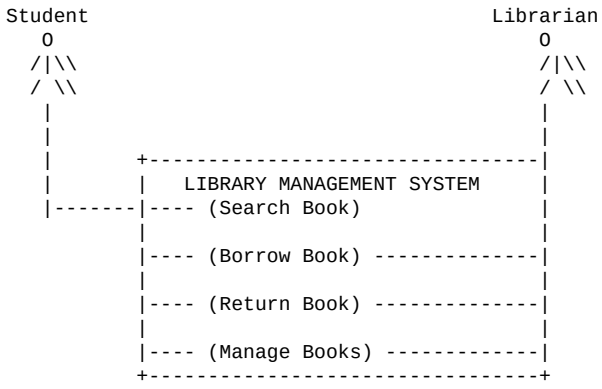
3. System Boundary

A rectangle that encloses the use cases belonging to the system.

4. Association

A line connecting an actor to a use case.

Example: Library Management System



How to draw this accurately in the exam:

• 1. Draw a large rectangle labelled `Library Management System`.

• 2. Draw two stick figures outside it: Student and Librarian.

• 3. Draw four ovals inside it: Search Book, Borrow Book, Return Book, and Manage Books.

• 4. Connect Student to Search Book, Borrow Book, and Return Book.

• 5. Connect Librarian to Manage Books.

Advantages

• 1. Easy to understand.

• 2. Helps identify functional requirements.

• 3. Defines the boundary of the system.

• 4. Improves communication between users and developers.

• 5. Helps prepare test cases from functional requirements.

Exam question: Explain the use case diagram with a suitable example.

15. Relationships Among Use Cases

UML provides three important relationships between use cases:

• 1. Include.

• 2. Extend.

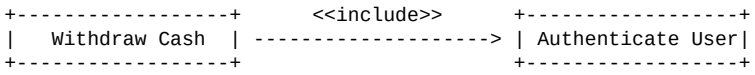
• 3. Generalization.

1. Include Relationship

The `<<include>>` relationship indicates that a use case always incorporates the behavior of another use case when that inclusion point is executed.

It is useful for extracting common functionality that is reused by several use cases.

Example: Both Withdraw Cash and Check Balance may include Authenticate User.



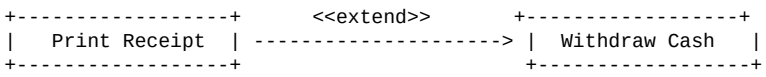
Remember: The dashed arrow points from the including use case toward the included use case.

2. Extend Relationship

The <<extend>> relationship represents additional, conditional behavior that may occur in a base use case.

The base use case can be complete without the extending behavior.

Example: An ATM withdrawal may optionally print a receipt.

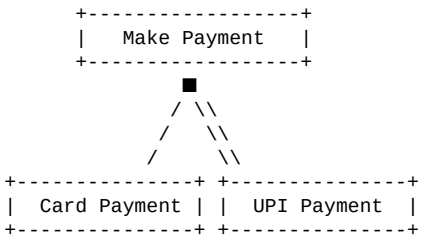


Remember: The dashed arrow points from the extending use case toward the base use case.

3. Generalization Relationship

Generalization represents a specialized use case or actor that inherits behavior from a more general one.

Example: Card Payment and UPI Payment can be specialized forms of Make Payment.



The hollow triangle points toward the general use case.

Difference Between Include and Extend

Include	Extend
Represents included behavior	Represents additional behavior
Used when the inclusion is part of the flow	Used for conditional or optional behavior
Helps reuse common functionality	Helps model optional extensions
Arrow points toward the included use case	Arrow points toward the base use case

Important: Include and extend are represented by dashed arrows labelled with the respective stereotype.

Exam question: Explain the relationships among use cases with suitable diagrams.

PART 5: INTERACTION DIAGRAMS

16. Interaction Diagrams

Definition

Interaction diagrams are UML diagrams that show how objects or participants communicate through messages to perform a particular task.

They help explain the dynamic behavior of a system.

Your syllabus includes two interaction diagrams:

• 1. Sequence Diagram.

• 2. Collaboration Diagram, also called a Communication Diagram in modern UML terminology.

Importance

• 1. Shows communication between objects.

• 2. Explains the order of operations.

• 3. Helps understand system behavior.

• 4. Supports detailed design.

• 5. Helps identify missing or incorrect interactions.

17. Sequence Diagram

Definition

A sequence diagram is a UML interaction diagram that shows the messages exchanged between participants in **time order**.

Time progresses from top to bottom.

Main Elements

1. Actor: An external user or system.

2. Participant/Object: An object involved in the interaction.

3. Lifeline: A vertical dashed line representing a participant over time.

4. Message: An arrow representing communication between participants.

5. Activation: A narrow rectangle on a lifeline showing when a participant is executing an operation.

6. Return Message: A message representing the result returned by an operation, often shown with a dashed arrow.

Example: User Login

Suppose a user logs into an application.

The interaction involves:

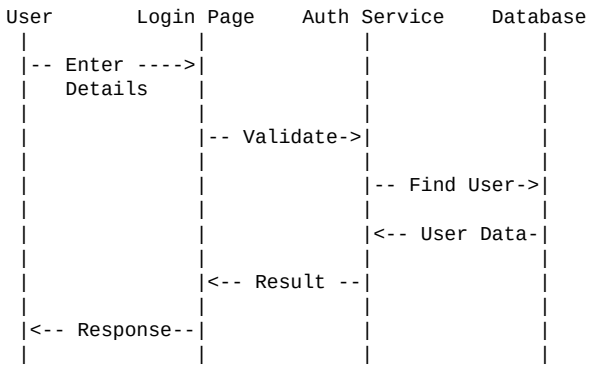
• User.

• Login Page.

• Authentication Service.

• Database.

Diagram



Note: The diagram is a simplified exam sketch. When drawing it, represent the participants' lifelines as vertical dashed lines and the messages as horizontal arrows.

Working

• 1. The user enters login credentials.

• 2. The login page sends the credentials to the authentication service.

• 3. The authentication service requests user information from the database.

• 4. The database returns the matching user data.

• 5. The authentication service validates the credentials.

• 6. The login page displays the success or failure response.

Advantages

• 1. Clearly shows message order.

• 2. Helps understand the flow of operations.

• 3. Identifies which object performs each operation.

• 4. Helps design APIs and object interfaces.

• 5. Useful for documenting complex interactions.

Applications

• User authentication.

• Payment processing.

• Online shopping.

• ATM transactions.

• API communication.

Exam question: Explain the sequence diagram with a suitable example.

18. Collaboration Diagram

Definition

A collaboration diagram, called a **communication diagram in modern UML**, represents interactions among objects by emphasizing their structural relationships and the messages exchanged.

Unlike a sequence diagram, which emphasizes time order, a collaboration diagram emphasizes how participating objects are connected.

Messages are numbered to indicate their sequence.

Main Elements

• 1. **Objects:** Participants in the interaction.

• 2. **Links:** Connections between participating objects.

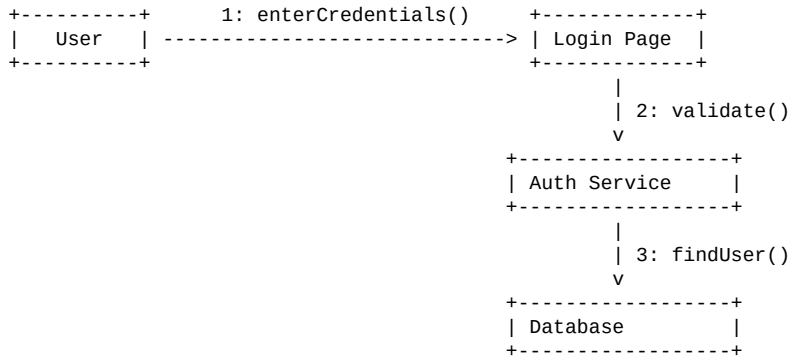
• 3. **Messages:** Operations or information sent between objects.

• 4. **Sequence numbers:** Indicate the order of messages.

Example: User Login

The same login scenario can be represented using a collaboration diagram.

Diagram



Working

• 1. The User sends login information to the Login Page.

• 2. The Login Page sends a validation request to the Authentication Service.

• 3. The Authentication Service requests user data from the Database.

• 4. The Database returns the required data.

• 5. The Authentication Service returns the validation result to the Login Page.

• 6. The Login Page displays the result to the User.

In a complete collaboration diagram, return messages can also be shown if required.

Advantages

• 1. Shows relationships between interacting objects.

• 2. Identifies which objects communicate with one another.

• 3. Numbered messages make the interaction order clear.

• 4. Helps identify the responsibilities of objects.

• 5. Useful for analyzing object collaboration.

Exam question: Explain the collaboration diagram with a suitable example.

19. Difference Between Sequence and Collaboration Diagrams

This is an important comparison question for semester examinations.

Sequence Diagram	Collaboration Diagram
Emphasizes the time order of messages	Emphasizes object relationships and message exchange
Uses vertical lifelines	Uses objects connected by links
Time progresses from top to bottom	Message order is indicated by sequence numbers
Clearly shows when messages occur	Clearly shows which objects communicate
Useful for analyzing temporal behavior	Useful for analyzing object collaboration
Example: Login sequence over time	Example: Login objects and their connections

Remember: Both diagrams can represent the same interaction, but they emphasize different aspects of it.

PART 6: IMPORTANT EXAM QUESTIONS

Based on your syllabus, prepare the following questions first.

Very Important 6-Mark Questions

No.	Question	Important diagrams
1	Explain software architecture and its architectural views.	4+1 view model
2	Explain the Pipe-and-Filter architectural style.	Pipe-and-filter diagram
3	Explain the Client-Server architectural style.	Client-server diagram
4	Explain the Shared-Data architectural style.	Shared repository diagram
5	Explain objects and object classes with examples.	Class and object diagram
6	Explain the Object-Oriented Design process.	Design process flowchart
7	Explain design evolution and its importance.	Design evolution flowchart
8	Explain the building blocks of UML.	UML building blocks

No.	Question	Important diagrams
9	Explain UML class diagrams and their elements.	Class diagram
10	Explain different class relationships in UML.	Association, aggregation, composition, generalization
11	Explain use case scenarios with an example.	Scenario steps
12	Explain use case diagrams.	Library management system
13	Explain relationships among use cases.	Include, extend, generalization
14	Explain sequence diagrams with an example.	Login sequence diagram
15	Explain collaboration diagrams with an example.	Login collaboration diagram
16	Differentiate between sequence and collaboration diagrams.	Comparison table

Most Important Diagrams to Practise

If you have limited preparation time, practise these seven diagrams first:

- 1. **4+1 Architectural Views**
- 2. **Pipe-and-Filter Architecture**
- 3. **Client-Server Architecture**
- 4. **UML Class Diagram**
- 5. **Use Case Diagram**
- 6. **Sequence Diagram**
- 7. **Collaboration Diagram**

Also learn the UML notations for association, aggregation, composition, generalization, dependency, realization, `<<include>>`, and `<<extend>>`.

PART 7: HOW TO WRITE A 6-MARK ANSWER

For most of these topics, follow this structure in the examination.

1. Definition — 1 mark

Write a clear definition in two or three sentences.

2. Diagram — 1 to 2 marks

Draw a neat, labelled diagram where appropriate.

3. Explanation — 2 marks

Explain the components and working in four to six points.

4. Advantages or applications — 1 mark

Write three or four relevant points.

The exact marking scheme depends on your university, so this is a preparation strategy rather than a guaranteed allocation of marks.

Final Revision Strategy

Study the unit in this order:

• 1. Pipe-and-Filter, Client-Server, and Shared Data.

• 2. Objects, classes, Object-Oriented Design, and design evolution.

• 3. UML building blocks and class diagrams.

• 4. Class relationships.

• 5. Use case scenarios, use case diagrams, and their relationships.

• 6. Sequence and collaboration diagrams.

One final tip: For UML questions, practise drawing the diagrams yourself instead of only memorizing the definitions. Correct notation and clear labels make your answers much easier to understand and evaluate.

Further Reference

For an authoritative introduction to UML, see the Object Management Group's UML resource: https://www.omg.org/uml/what-is-uml.htm

Microsoft's architecture guidance also provides a useful explanation of the Pipe-and-Filter style: https://learn.microsoft.com/en-us/azure/architecture/patterns/pipes-and-filters